Undergraduate & Postgraduate Course Catalog AY 2025-2026

School of Computing & Engineering | B.Tech & M.Tech CSE Programme Specifications

1. B.Tech CSE Degree Architecture & CBCS Curriculum Framework

The Bachelor of Technology (B.Tech) in Computer Science and Engineering at Northgate Institute is an 8-semester, 4-year professional engineering programme structured under the Choice-Based Credit System (CBCS) in accordance with AICTE Model Curriculum 2022 and NBA Tier-1 accreditation criteria. It requires a minimum of 160 academic credits distributed across seven curricular categories. The programme is aligned with the Washington Accord Graduate Attributes and prepares students for careers in software engineering, data science, systems architecture, AI/ML research, cloud computing, and entrepreneurship.

- **Basic Sciences (BSC — 24 Credits):** Engineering Mathematics I (Linear Algebra & Calculus), Mathematics II (ODE & Complex Analysis), Mathematics III (Probability, Statistics & Transforms), Mathematics IV (Numerical Methods & Graph Theory), Engineering Physics with Lab, Engineering Chemistry with Lab.
- **Engineering Sciences (ESC — 20 Credits):** Basic Electrical & Electronics Engineering, Engineering Graphics & Product Design using CAD, Programming Fundamentals with Python, Manufacturing Processes (Workshop Practice), Digital Logic & Circuit Design.
- **Humanities, Social Sciences & Management (HSMC — 12 Credits):** Professional Communication & Technical Writing, Environmental Science & Sustainability, Ethics, Human Values & IPR, Technology Entrepreneurship & Business Models.
- **Professional Core Courses (PCC — 64 Credits):** Data Structures, Algorithms, Computer Architecture, Discrete Mathematics, Database Systems (CS301), Operating Systems, Computer Networks, Automata Theory & Compilers, Machine Learning, Software Engineering & System Design — 16 courses across Semesters III–VII.
- **Professional Electives (PEC — 20 Credits):** 5 electives from a designated specialization track (AI/ML, Cyber Security, or Cloud & Distributed Systems) — Semesters VI–VIII.
- **Open Electives (OEC — 8 Credits):** 2 interdisciplinary courses from offered management, economics, cognitive sciences, or law schools.
- **Project Work, Internship & Seminars (PROJ — 12 Credits):** Mini-Project (Sem V, 2 Cr), Industrial Internship (Sem VI, 2 Cr), Capstone Major Project Phase I (Sem VII, 4 Cr), Capstone Major Project Phase II (Sem VIII, 4 Cr).

2. Semester I & II — Foundation Engineering Sciences

Semesters I and II build rigorous mathematical, computational, and engineering foundations. The curriculum integrates theory with laboratory practice to develop algorithmic thinking, circuit understanding, professional communication, and engineering drawing skills.

MA101 Engineering Mathematics I — Linear Algebra & Calculus (4 Credits | 3-1-0)

Vector spaces, subspaces, basis and dimension, linear transformations, eigenvalues and eigenvectors, diagonalization, singular value decomposition. Multivariable calculus: partial derivatives, gradient, Jacobian, Taylor expansion, Lagrange multipliers. Multiple integrals, line and surface integrals, Green's Theorem, Stokes' Theorem, Divergence Theorem. Application to computer graphics, machine learning (PCA, SVD), and network analysis.

CS101 Programming Fundamentals with Python (4 Credits | 3-0-2)

Computational thinking, algorithmic problem decomposition, Python 3 syntax: variables, types, control flow (if/elif/else, for, while), function definition and scope, recursion and memoization, Python data structures (list, tuple, dict, set, frozenset), comprehensions, generators and iterators, file I/O (text, CSV, JSON), exception handling, modules and packages, OOP fundamentals (classes, inheritance, polymorphism, dunder methods). Introduction to NumPy and Matplotlib for scientific computing. Lab: 15 practical exercises covering number theory, string algorithms, sorting and searching, matrix operations, and data visualization.

3. Semester III & IV — Core CS Foundations

CS201 Data Structures & Algorithm Design (4 Credits | 3-1-2)

Sequential structures: arrays (multi-dimensional, jagged), singly/doubly/circular linked lists, stacks (balanced parentheses, expression evaluation), queues (circular, deque, priority queue). Non-linear structures: binary trees, BST operations (insert, delete, traversal), AVL trees (rotation cases, balance factor), Red-Black trees, heaps (heapify, heapsort, priority queues), hash tables (separate chaining, open addressing, load factor, universal hashing), tries. Graphs: adjacency matrix vs list, BFS, DFS, topological sort (DFS-based and Kahn's), shortest paths (Dijkstra, Bellman-Ford, Floyd-Warshall), minimum spanning trees (Kruskal's with DSU, Prim's with priority queue). Algorithm paradigms: divide and conquer, dynamic programming (coin change, LCS, edit distance, 0-1 knapsack, matrix chain), greedy (activity selection, Huffman encoding). Amortized analysis. NP-completeness introduction.

CS210 Discrete Mathematical Structures (4 Credits | 3-1-0)

Propositional and predicate logic, logical equivalences, proof methods: direct, indirect, proof by contradiction, mathematical induction (strong induction), well-ordering principle. Set theory: operations, power sets, Cartesian product, Venn diagrams. Relations: reflexive, symmetric, transitive, equivalence relations, partial orders, Hasse diagrams. Functions: injective, surjective, bijective, composition, inverse. Combinatorics: pigeonhole principle, permutations, combinations, inclusion-exclusion, generating functions, recurrence relations (linear homogeneous, Fibonacci, Catalan numbers). Graph theory: Eulerian and Hamiltonian paths, graph coloring (chromatic number, four-color theorem), planar graphs, trees and spanning trees. Algebraic structures: groups, rings, fields, lattices, Boolean algebras.

CS240 Computer Organization & Architecture (4 Credits | 3-0-2)

Number representations: two's complement, IEEE 754 floating point, signed overflow. ISA design: MIPS and RISC-V instruction formats, addressing modes, calling conventions, ABI and stack frames. Datapath design: ALU, multiplexers, register file, single-cycle vs pipelined implementation. Pipeline hazards: data hazards (forwarding, stalls), structural hazards, control hazards (branch prediction: always-taken, two-bit predictor, BHT, BTB). Memory hierarchy: SRAM vs DRAM, direct-mapped, N-way set-associative and fully-associative cache, write-back vs write-through, cache coherence (MESI protocol). Virtual memory: TLBs, multi-level page tables (x86-64 4-level), page fault handling. I/O: programmed I/O, interrupt-driven I/O, DMA controller operation. Lab: MIPS assembly programming, Logisim circuit simulation, cache performance analysis using valgrind.

4. Semester V — Core Professional Track

CS301 Database Management Systems (4 Credits | 3-1-2)

Unit I: File systems vs DBMS, three-schema architecture, data independence, relational/hierarchical/network models. ER modeling: entity sets, attributes (composite, multivalued, derived), relationship sets (cardinality: 1:1, 1:N, M:N), weak entities, identifying relationships, extended ER (specialization, generalization, aggregation), 7-step ER-to-relational mapping. Unit II: Relational algebra (select σ, project π, rename ρ, cross product ×, natural join ■, theta join, outer joins, division ÷). SQL DDL (CREATE, ALTER, DROP, constraints), DML (SELECT with subqueries, correlated subqueries, EXISTS, aggregates, GROUP BY, HAVING), window functions (RANK, DENSE_RANK, ROW_NUMBER, LAG, LEAD), CTEs, views, triggers, stored procedures. Unit III: Functional dependencies (FD), Armstrong's axioms (reflexivity, augmentation, transitivity), attribute closure, canonical covers, normal forms (1NF, 2NF, 3NF, BCNF), lossless-join decomposition, dependency-preservation, Bernstein's 3NF synthesis, multi-valued dependencies, 4NF. Unit IV: Transactions, ACID properties, concurrency anomalies (dirty read, non-repeatable read, phantom read), conflict serializability, precedence graph, recoverability, cascadeless schedules, Two-Phase Locking (strict 2PL, rigorous 2PL), timestamp ordering, MVCC, deadlock detection (wait-for graph), prevention (wound-wait, wait-die), ARIES recovery: write-ahead logging, checkpoint, analysis pass, redo pass, undo pass. Unit V: Physical storage (heap files, sorted files, clustered/unclustered), RAID levels (0, 1, 4, 5, 6, 1+0), B+ tree structure (internal vs leaf nodes, insertion/deletion algorithms, bulk loading), hash-based indexes, bitmap indexes. Heuristic query optimization: equivalence rules, selection/projection push-down, join ordering, statistics-based cost estimation. Lab: PostgreSQL schema design, SQL performance tuning with EXPLAIN ANALYZE, B+ tree insertion benchmarking, transaction isolation level experiments.

CS340 Operating Systems Architecture (4 Credits | 3-1-2)

Monolithic vs microkernel vs hybrid OS designs. Process lifecycle, PCB, process creation (fork/exec), context switching overhead analysis. Threading: kernel threads vs user threads, POSIX pthreads, thread pools. CPU scheduling: FCFS, SJF (preemptive SRTF), Round Robin (quantum selection trade-offs), Priority Scheduling, Multilevel Feedback Queue (MLFQ). Synchronization: Race conditions, critical section problem, Peterson's solution, hardware atomics (TestAndSet, CompareAndSwap), spinlocks, semaphores (binary, counting), mutex locks, monitors, condition variables. Classical problems: Bounded Buffer, Readers-Writers (3 variants), Dining Philosophers (Chandy-Misra solution). Deadlocks: 4 Coffman conditions, RAG, Banker's safety algorithm, detection & recovery. Memory: logical vs physical address spaces, MMU, segmentation, paging, multi-level page tables, inverted page tables, TLB shootdown, demand paging, page replacement (FIFO — Belady's anomaly, Optimal, LRU via stack/clock approximation), thrashing and working set model. File systems: VFS, inode structure, directory traversal, allocation methods, fsck recovery. I/O: device drivers, interrupt handling, disk scheduling (SSTF, SCAN, C-SCAN, LOOK). Virtualization: Type-1 (bare-metal) and Type-2 hypervisors, hardware-assisted virtualization (VT-x, AMD-V), container isolation via Linux namespaces and cgroups, Docker internals.

CS355 Computer Networks & Internet Protocols (4 Credits | 3-0-2)

OSI vs TCP/IP 5-layer model. Physical layer: Nyquist/Shannon theorems, encoding schemes, fiber vs copper. Data link: framing, CRC polynomial division, sliding window protocols (Stop-and-Wait, Go-Back-N, Selective Repeat), MAC sub-layer, Ethernet CSMA/CD, 802.11 CSMA/CA, spanning tree protocol. Network layer: IPv4 addressing and subnetting (VLSM, CIDR), IPv6 (128-bit, stateless autoconfiguration), ARP, ICMP, NAT. Routing: Dijkstra OSPF (link-state), Bellman-Ford RIP (distance vector), BGP (path vector, AS policies), MPLS labels. Transport: TCP three-way handshake, state machine, reliable delivery via retransmission, congestion control (slow start, congestion avoidance, AIMD, fast retransmit, Reno, CUBIC, BBR), UDP checksum. Application: HTTP/1.1 vs HTTP/2 vs HTTP/3 (QUIC), DNS hierarchy and resolution, SMTP/IMAP, TLS 1.3 handshake, CDN architecture. Lab: Wireshark packet analysis, socket programming (TCP/UDP client-server), mininet network emulation.

CS360 Automata Theory & Compiler Design (4 Credits | 3-1-0)

Part A — Formal Languages: DFAs and NFAs (subset construction, Myhill-Nerode minimization), regular expressions (Kleene's theorem, Arden's equation), pumping lemma for regularity, closure properties of regular languages. CFGs: derivation, parse trees, ambiguity, simplification (CNF, GNF), pumping lemma for CFLs, closure properties. PDAs (NPDA from CFG, determinism), Turing Machines (variants: multi-tape, non-deterministic), decidability, the Halting Problem, Rice's Theorem, reducibility, complexity classes P and NP. Part B — Compilers: Lexical analysis (maximal munch, Flex tool, DFA-based scanner), syntax analysis: predictive parsing (FIRST/FOLLOW, LL(1) table construction), bottom-up parsing (LR(0) items, SLR(1), LALR(1), CLR(1)), error recovery strategies. Semantic analysis: symbol table management, type checking (Hindley-Milner inference), scope rules. Intermediate code generation: three-address code (TAC), SSA form, AST lowering. Code optimization: constant folding, dead-code elimination, loop-invariant code motion, register allocation (graph coloring). Target code generation for RISC-V.

5. Semester VI — Advanced Core & System Design

CS420 Applied Machine Learning & AI (4 Credits | 3-0-2)

Unit I: Supervised learning — hypothesis space, empirical risk minimization, Linear Regression (OLS, Lasso L1, Ridge L2, ElasticNet), feature engineering, polynomial features, bias-variance tradeoff, cross-validation (k-fold, stratified), hyperparameter tuning (grid search, Bayesian optimization). Logistic Regression (sigmoid, softmax, multi-class OvR/OvO). Unit II: Decision Trees (Gini impurity, information gain, entropy, CART algorithm, pruning), Random Forests (bagging, feature sub-sampling, variable importance), Gradient Boosting (functional gradient descent, XGBoost, LightGBM, CatBoost), SVMs (hard/soft margin, kernel trick: RBF, polynomial, sigmoid, Mercer's condition). Unit III: Neural Networks — perceptron, MLP, activation functions (sigmoid, tanh, ReLU, Leaky ReLU, GELU, Swish), backpropagation derivation via chain rule, weight initialization (Xavier, He), optimizers (SGD, Momentum, RMSProp, Adam, AdamW), batch normalization, dropout, L2 regularization. Deep learning: CNNs (convolution, pooling, receptive field, ResNet residual connections, batch norm), RNNs (vanishing gradient problem, LSTM gates, GRU), Transformer architecture (scaled dot-product attention, multi-head attention, positional encoding, BERT, GPT), LoRA fine-tuning for LLMs. Unit IV: Unsupervised learning — K-Means++ initialization, Elbow method, DBSCAN (epsilon-neighborhood, MinPts, noise points), hierarchical clustering (Ward linkage), PCA (covariance eigendecomposition, scree plot), t-SNE (KL divergence optimization), UMAP, Gaussian Mixture Models (EM algorithm). Unit V: Evaluation metrics, confusion matrix, ROC-AUC, PR curve, F1, MCC, calibration. Responsible AI: fairness metrics (demographic parity, equalized odds), explainability (SHAP, LIME, Integrated Gradients), data governance, model cards. Lab: Python (scikit-learn, PyTorch) — end-to-end ML pipelines, CNN image classification, LLM prompt engineering and fine-tuning workshop.

CS460 Software Engineering & System Design (4 Credits | 3-0-2)

Part 1 — Low-Level Design (LLD): SDLC models (Waterfall, Agile Scrum, Kanban, XP), OOP principles (encapsulation, abstraction, inheritance, polymorphism), SOLID principles (SRP, OCP, LSP, ISP, DIP) with anti-pattern examples. Design Patterns (GoF 23): Creational (Thread-safe Singleton, Factory Method, Abstract Factory, Builder, Prototype), Structural (Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy), Behavioral (Strategy, Observer Pub-Sub, Command Undo/Redo, Template Method, Iterator, State Machine, Mediator, Chain of Responsibility). UML diagrams: Class, Sequence, Activity, State, Component, Deployment. Case study: Enterprise Parking Lot System — multi-gate architecture, vehicle types (two-wheeler, four-wheeler, bus), smart pricing strategies, reservations. Part 2 — High-Level Design (HLD): Scalability (vertical vs horizontal), load balancing (L4 vs L7, round-robin, least-connections, consistent hashing, rendezvous hashing). Storage: SQL vs NoSQL decision framework, replication (leader-follower, multi-leader, leaderless), database sharding (range, hash, directory-based), connection pooling. Caching: Cache-aside, Read-through, Write-through, Write-behind, LRU/LFU eviction, Redis clusters, CDN edge caching. CAP theorem (formal proof), PACELC extension, BASE properties. Message queues: Apache Kafka (topics, partitions, consumer groups, ISR), RabbitMQ. Microservices: bounded contexts (DDD), API gateway (rate limiting — token bucket, leaky bucket; circuit breaker — Hystrix/Resilience4j), service mesh (Envoy, Istio). Distributed transactions: 2PC, SAGA (orchestration vs choreography). Consensus: Raft (leader election, log replication, safety guarantee). End-to-end system design case studies: URL shortener (TinyURL — 100M daily writes, Base62, KGS), push notification system (WebSocket, FCM/APNS, deduplication), distributed online exam proctoring platform (auto-scaling, WebRTC, anti-cheating AI).

6. Professional Elective Tracks & Course Offerings (Semesters VI–VIII)

Track A — AI & Intelligent Systems: (1) CS501 Natural Language Processing: tokenization, word embeddings (Word2Vec, GloVe, FastText), seq2seq, BERT pre-training, GPT architecture, retrieval-augmented generation (RAG), knowledge graphs. (2) CS502 Computer Vision: image formation, edge detection, HOG features, CNN architectures (AlexNet, VGG, ResNet, EfficientNet), object detection (YOLO v8, DETR, SAM), generative models (VAE, GAN, Stable Diffusion). (3) CS503 Reinforcement Learning: MDP formulation, Bellman equations, Q-learning, DQN, policy gradient (REINFORCE, PPO, SAC).

Track B — Systems & Cybersecurity: (1) CS511 Network Security & Cryptography: symmetric encryption (AES-256, ChaCha20), asymmetric (RSA, ECC), hash functions (SHA-3), TLS internals, PKI, digital certificates, zero-trust architecture, firewall design, IDS/IPS. (2) CS512 Blockchain & Decentralized Systems: consensus mechanisms (PoW, PoS, dBFT, Tendermint), smart contracts (Solidity, EVM), DeFi primitives, supply-chain provenance. (3) CS513 Penetration Testing & Cyber Forensics: OWASP Top-10 web vulnerabilities, Metasploit, reverse engineering, memory forensics, incident response.

Track C — Cloud & Distributed Computing: (1) CS521 Cloud Architecture: AWS/GCP/Azure services comparison, IaaS/PaaS/SaaS, Kubernetes orchestration (pods, deployments, services, ingress, HPA), Terraform IaC, serverless (Lambda/Cloud Functions). (2) CS522 Distributed Databases: BigTable, Spanner, DynamoDB, CockroachDB, TiDB, vector databases (Pinecone, Qdrant, Weaviate) for RAG applications. (3) CS523 Edge Computing & IoT: edge inference optimization (model quantization, pruning, knowledge distillation), MQTT, fog computing frameworks, real-time data pipelines.

7. Academic Progression, Prerequisite Policy & Credit Overload

Students must pass all listed prerequisites with a minimum grade of 'C' (5.0 grade points) before enrolling in dependent upper-division courses. Standard maximum semester registration is 24 credits. Students maintaining a cumulative CGPA >= 8.50 with no active backlogs may petition for a Credit Overload up to 28 credits to pursue the Honors Research stream or dual elective specialization. Students carrying more than 12 credit-hours of active backlogs are administratively limited to 18 credits in the subsequent semester. The Honors Thesis Programme (CS490H, 6 credits, replacing Capstone Phase II) requires written endorsement from a faculty supervisor with active research funding and approval from the Department Research Committee.

8. M.Tech Computer Science & Engineering Programme Overview

The M.Tech programme (2-year, 4 semesters, 72 credits) admits students through GATE CS/DA scores, with a minimum qualifying GATE percentile of 85. Specializations: AI & Machine Learning, Software Systems & Security, and Data Engineering & Cloud Computing. The programme requires: 8 core theory courses (32 Cr), 4 elective courses (16 Cr), a 2-semester research thesis (20 Cr), and 1 seminar (4 Cr). Thesis evaluation includes an open seminar before submission and final viva voce with 2 external examiners.